

Лекция 10. Балансировка нагрузки и доступность web-сервисов с HAProxy

HAProxy принимает клиентский трафик и направляет его на один из backend-серверов

Балансировщик помогает убрать единую точку отказа web-сервера, но сам тоже требует отказоустойчивости

Учебный стенд: client01 → lb01 → web01 и web02

Главная идея: балансировка работает только тогда, когда checks, таймауты, logs, TLS и мониторинг настроены осознанно

Цель лекции

Понять SPOF, вертикальное и горизонтальное масштабирование

Уметь различать Reverse Proxy и Load Balancer

Уметь объяснить frontend, backend, server, global, defaults и mode http

Уметь настроить Round Robin, Least Connections, weights и HTTP health checks

Уметь проверить отказ backend, TLS termination, stats и мониторинг HAProxy

Учебные вопросы

1. SPOF, вертикальное и горизонтальное масштабирование
2. Reverse proxy и load balancer
3. HAProxy frontend, backend и server
4. Round Robin, Least Connections и health checks
5. Таймауты, исходный IP и статистика
6. Отказ backend, TLS termination, один HAProxy и обзор Keepalived/VIP
7. Мониторинг HAProxy через Zabbix

Учебная топология фиксирует роли и адреса

client01: 192.168.10.10 — клиентская проверка

lb01: 192.168.30.10 — HAProxy

web01: 192.168.30.11 — Nginx backend

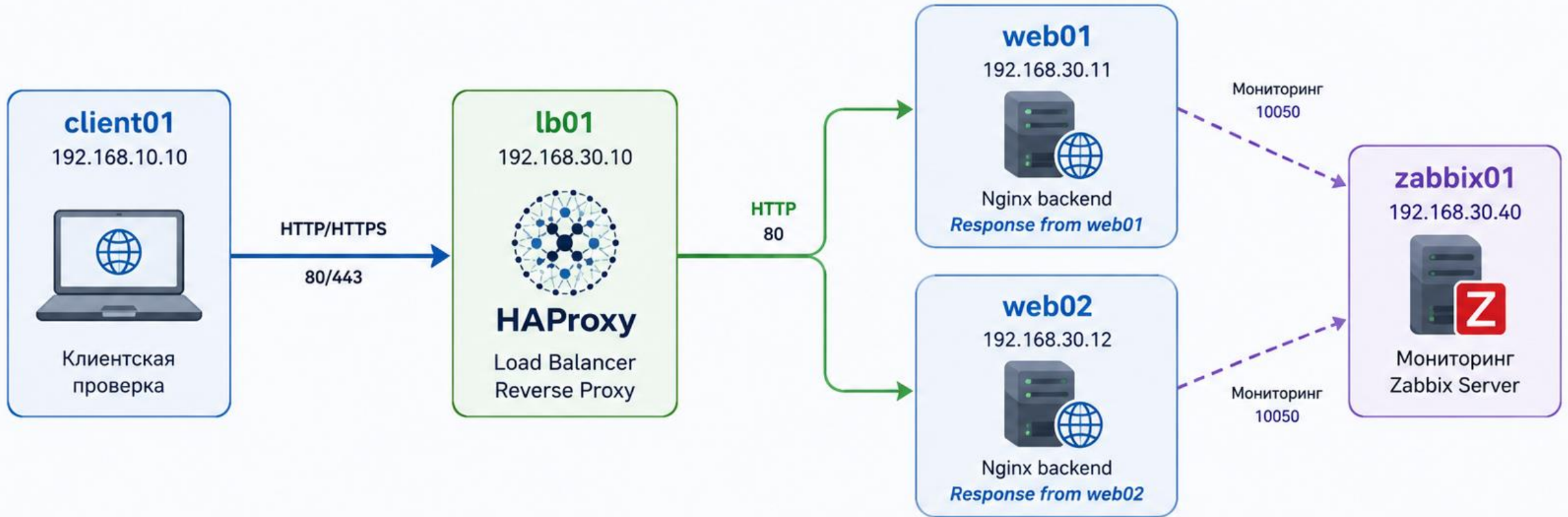
web02: 192.168.30.12 — Nginx backend

zabbix01: 192.168.30.40 — мониторинг

На web01 и web02 нужны разные страницы: Response from web01 и Response from web02

Топология: client01, lb01, web01, web02 и zabbix01

Учебный стенд: клиент отправляет запросы на HAProxy (lb01), который балансирует трафик между backend-серверами web01 и web02.



Роли	Сети
client01 – клиентская проверка	— 192.168.10.0/24 – клиентская сеть
lb01 – HAProxy (балансировка)	— 192.168.30.0/24 – серверная сеть
web01 – Nginx backend	
web02 – Nginx backend	
zabbix01 – мониторинг	

Поток запросов
→ Клиент → HAProxy (HTTP/HTTPS 80/443)
→ HAProxy → backend (HTTP 80)
- - - Мониторинг (Zabbix agent 10050)

Важно
✓ На web01 и web02 должны быть разные страницы: Response from web01 и Response from web02
✓ HAProxy распределяет трафик между backend-серверами (Round Robin по умолчанию)

1. Один web-сервер становится SPOF

SPOF — Single Point of Failure, единая точка отказа

Если единственный web-сервер отказал, сервис недоступен всем клиентам

Резервная копия помогает восстановиться позже, но не обеспечивает доступность прямо сейчас

Мониторинг обнаружит отказ, но не распределит трафик

Балансировщик нужен, чтобы клиент мог попасть на исправный backend

Вертикальное масштабирование усиливает один сервер

Вертикальное масштабирование — добавить CPU, RAM, диск или сетевой ресурс одному серверу

Плюс: проще администрировать один экземпляр приложения

Минус: сохраняется одна точка отказа

Минус: есть предел роста одного сервера

Для отказоустойчивости одного усиления обычно недостаточно

Горизонтальное масштабирование добавляет backend-серверы

Горизонтальное масштабирование — несколько серверов обслуживают один сервис

Нагрузка распределяется между web01 и web02

При отказе одного backend второй может продолжить обслуживание

Приложение должно быть готово к работе в нескольких экземплярах

Балансировщик выбирает, куда отправить конкретный запрос

HAProxy не исправляет состояние приложения автоматически

Если оба backend зависят от одной базы данных, database остается SPOF

Если backend имеют разные версии приложения, Round Robin будет отдавать разные результаты

Если session хранится локально на web01, клиент может потерять авторизацию при попадании на web02

Если uploaded files лежат только на одном backend, второй backend не сможет их отдать

Балансировка требует согласованной архитектуры приложения

2. Reverse Proxy и Load Balancer

Reverse Proxy принимает запрос вместо внутреннего сервера и пересылает его дальше

Load Balancer выбирает один из нескольких backend

Reverse Proxy может обслуживать один backend и не выполнять балансировку

Load Balancer обычно реализуется как reverse proxy или L4 proxy

HAProxy может выполнять обе роли одновременно

Reverse Proxy без балансировки

Схема: Client → HAProxy → web01

HAProxy скрывает внутренний адрес web01

Можно централизовать TLS, headers, ACL и logging

Но если web01 отказал, второго backend нет

Это reverse proxy, но еще не отказоустойчивая
балансировка

Reverse Proxy с балансировкой нагрузки

Схема: Client → HAProxy → web01 или web02

HAProxy принимает один внешний адрес сервиса

Backend-серверы остаются во внутренней сети

Health checks исключают неисправный backend из pool

Клиент продолжает обращаться к одному имени или

IP балансировщика

3. HAProxy frontend, backend и server

global задает общие параметры процесса HAProxy

defaults задает параметры по умолчанию для frontend и backend

frontend принимает клиентские соединения

backend описывает pool серверов назначения

server — конкретный backend-узел внутри backend-секции

mode http нужен для HTTP-лаборатории

HAProxy может работать в mode tcp и mode http

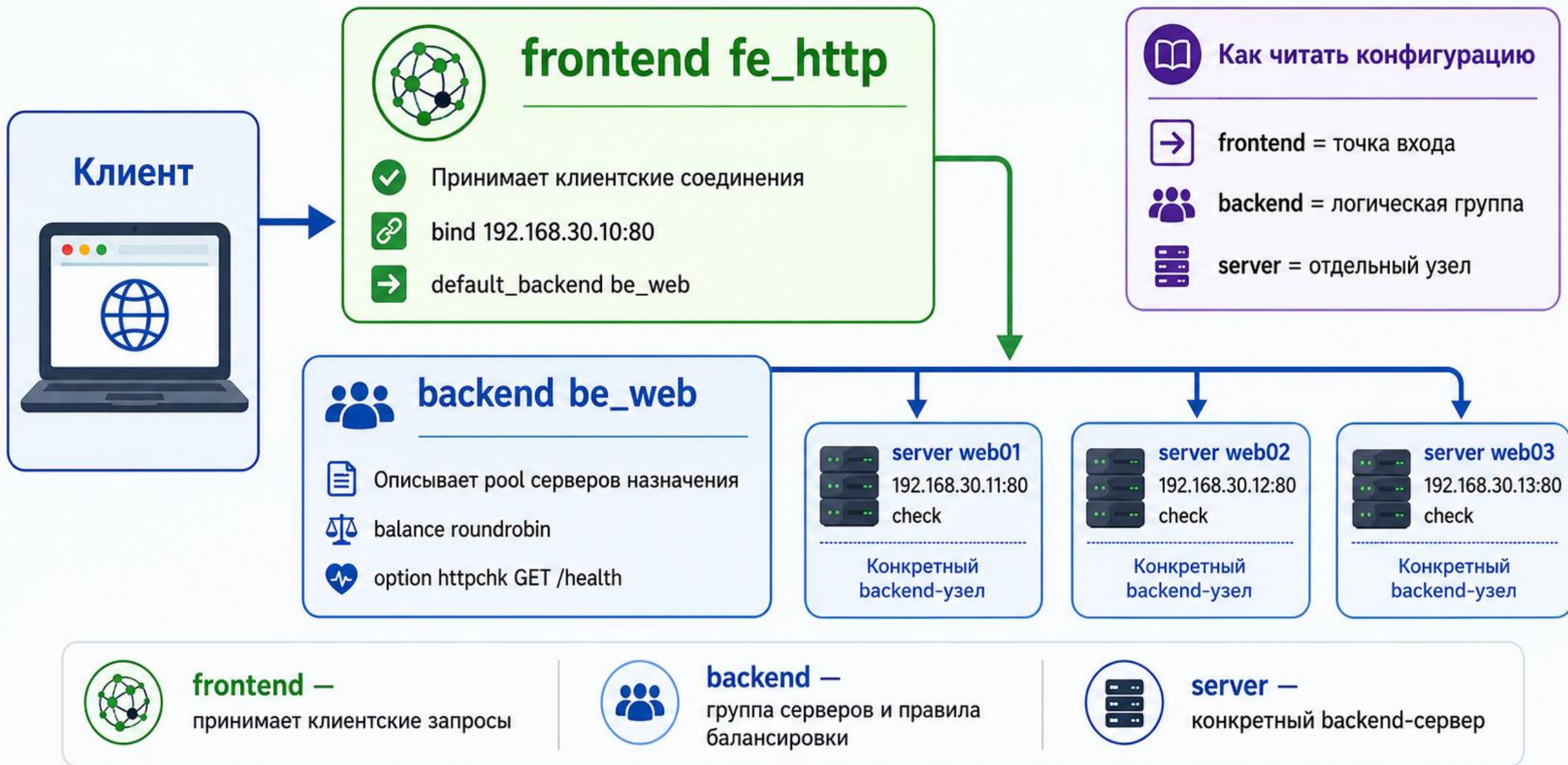
mode tcp работает на уровне TCP-соединения

mode http понимает HTTP-заголовки, URL, Host Header и status codes

Для option forwardfor, HTTP health check и routing по Host нужен mode http

В лаборатории mode http задается в defaults

Схема: frontend, backend и server в HAProxy



Минимальный haproxy.cfg: global

global

log /dev/log local0

log /dev/log local1 notice

user haproxy

group haproxy

daemon

Минимальный haproxy.cfg: defaults

defaults

log global

mode http

option httplog

option dontlognull

timeout connect 5s

timeout client 30s; timeout server 30s

Минимальный haproxy.cfg: frontend

```
frontend fe_http
  bind 192.168.30.10:80
  http-request del-header X-Forwarded-For
  http-request add-header X-Forwarded-For %[src]
  default_backend be_web
```

bind *:80 слушает все адреса; в лаборатории лучше
указать конкретный IP

Минимальный haproxy.cfg: backend

```
backend be_web
```

```
    balance roundrobin
```

```
    option httpchk GET /health
```

```
    http-check expect status 200
```

```
    server web01 192.168.30.11:80 check inter 2s fall 3 rise
```

```
2
```

```
    server web02 192.168.30.12:80 check inter 2s fall 3 rise
```

```
2
```

bind должен соответствовать сетевой роли интерфейса

bind *:80 открывает порт на всех подходящих адресах хоста

Для учебного service IP используем bind
192.168.30.10:80

Для stats и management используют отдельный
management IP

Проверка прослушивания: ss -ltnp | grep :80

Доступ также должен контролироваться firewall

4. Round Robin, Least Connections и health checks

Round Robin распределяет запросы по очереди между backend

Least Connections выбирает backend с меньшим числом активных соединений

Weights позволяют учитывать разную мощность backend

Health Check определяет, считается ли backend готовым принимать трафик

Проверка распределения должна учитывать HTTP Keep-Alive

Round Robin нужно проверять с закрытием соединения

HTTP Keep-Alive может отправить несколько запросов по одному соединению на один backend

Для лабораторной проверки полезно закрывать соединение после запроса

Команда: `for i in {1..10}; do curl -s -H 'Connection: close' http://192.168.30.10/; done`

web01 должен возвращать Response from web01

web02 должен возвращать Response from web02

Least Connections не измеряет полную нагрузку сервера

Least Connections учитывает активные соединения

Меньше соединений не всегда означает меньше CPU, RAM или очередей приложения

Для разных по мощности backend используют weights

Пример: server web01 192.168.30.11:80 check weight 100

Пример: server web02 192.168.30.12:80 check weight 50

TCP check не доказывает здоровье приложения

Строка `server web01 192.168.30.11:80` check может подтвердить только доступность TCP-порта

Nginx может слушать TCP 80, но приложение за ним возвращает HTTP 500

Для HTTP-сервиса нужен HTTP health check

option `httpchk GET /health` проверяет URL `/health`

`http-check expect status 200` требует ожидаемый HTTP-код

Health endpoint должен быть легким и осмысленным

L4 check проверяет доступность TCP-порта

HTTP check проверяет HTTP-ответ на конкретный URL

Application check может учитывать доступность
важных зависимостей

Liveness показывает, что процесс способен работать

Readiness показывает, что экземпляр готов принимать
пользовательский трафик

Для базовой лаборатории достаточно HTTP 200 на
/health

inter, fall и rise защищают от flapping

inter 2s — выполнять health check каждые 2 секунды

fall 3 — отметить backend DOWN после трех
неуспешных проверок

rise 2 — вернуть backend UP после двух успешных
проверок

Задержка возврата backend после запуска Nginx
является нормальной

Эти параметры снижают случайные переключения
UP/DOWN

Состояние session влияет на балансировку

Если session хранится локально на web01, клиент может потерять авторизацию на web02

Если uploaded file записан только на локальный диск web01, web02 не сможет его отдать

Если cache не синхронизирован, backend могут отвечать по-разному

Решения: stateless application, database, Redis session store, shared storage или replication

Backend должны иметь одинаковые версии приложения и конфигурации

Sticky sessions — компромисс, а не идеальная архитектура

Sticky sessions закрепляют клиента за выбранным backend

Методы: cookie-based persistence или source IP hash

Плюс: локальная session реже теряется между backend

Минусы: неравномерная нагрузка и сложность масштабирования

При отказе backend локальная session всё равно теряется

5. Таймауты, исходный IP и статистика

`timeout connect` ограничивает установление соединения с `backend`

`timeout client` ограничивает бездействие клиента

`timeout server` ограничивает ожидание данных от `backend`

Слишком короткий `timeout server` может оборвать законный отчет или загрузку

HAProxy обычно предупреждает о важных отсутствующих `timeout` при проверке конфигурации

Рабочий блок timeout для HTTP-лаборатории

timeout connect 5s

timeout client 30s

timeout server 30s

Для WebSocket и длительных запросов нужны другие значения

Изменение timeout требует функциональной проверки приложения

После reload проверяют журналы и пользовательский HTTP-запрос

X-Forwarded-For требует доверенной цепочки проху

Клиент может сам отправить поддельный X-Forwarded-For

Пример атаки: X-Forwarded-For: 127.0.0.1

HAProxy должен удалить недоверенный заголовок от клиента

Затем HAProxy добавляет проверенный адрес из %[src]

Приложение должно доверять только известному HAProxy как Trusted Proxy

X-Forwarded-Proto нужен после TLS termination

После TLS termination backend часто получает обычный HTTP

Приложение может ошибочно считать запрос незашифрованным

Команда: `http-request set-header X-Forwarded-Proto https if { ssl_fc }`

Иначе возможны неправильные redirects и HTTP-ссылки

Также возможны cookie без Secure и циклическое перенаправление

PROXY Protocol — альтернатива для передачи client IP

PROXY Protocol часто применяют в L4 proxy-сценариях

Пример: server web01 192.168.30.11:80 send-proxy

Backend должен явно поддерживать и ожидать PROXY Protocol

Если обычный web-сервер не ожидает PROXY Protocol, он увидит некорректный HTTP-запрос

Для текущей HTTP-лаборатории X-Forwarded-For проще

Stats page нельзя оставлять публичной

Basic Auth без HTTPS не является достаточной защитой

Stats лучше вынести на management IP

Доступ ограничивают firewall и management network

Для лаборатории достаточно read-only наблюдения

Административные действия через stats включают только при явной необходимости

Пример защищенного listen stats

listen stats

bind 192.168.40.10:8404

mode http

stats enable

stats uri /stats

stats auth monitor:CHANGE_ME; stats hide-version;

stats refresh 10s

6. Отказ backend, TLS termination и HA балансировщика

Остановка Nginx моделирует отказ процесса или TCP-порта

HTTP 500 при открытом порте моделирует прикладной отказ

Остановка обоих backend должна привести к 503 Service Unavailable

TLS termination переносит HTTPS на HAProxy

Один HAProxy тоже является SPOF и требует отдельной отказоустойчивости

Проверка отказа должна включать разные сценарии

Nginx остановлен на web01 — TCP или HTTP check переводит backend DOWN

/health возвращает 500 — TCP check может не заметить, HTTP check должен заметить

Firewall блокирует lb01 → web01 — backend становится DOWN

web01 отвечает медленно — проверяются timeout

Контент web01 отличается от web02 — выявляется несогласованность deployment

Полный отказ backend должен давать понятный результат

Один backend DOWN — сервис деградировал, но продолжает отвечать

Все backend DOWN — сервис недоступен

Проверка: `curl -i http://192.168.30.10/`

Ожидаемый результат при полном отказе: HTTP 503

Service Unavailable

Zabbix должен различать деградацию и полный отказ

Graceful drain нужен для планового обслуживания

Остановка Nginx — грубая имитация аварии

При обслуживании backend сначала выводят из pool

Затем ждут завершения активных соединений

После обновления backend проверяют локально и возвращают в pool

Для базовой лекции достаточно понимать идею drain или maintenance

TLS termination требует полноценного PEM-файла

```
frontend fe_https
```

```
    bind 192.168.30.10:443 ssl crt
```

```
    /etc/haproxy/certs/site.pem
```

```
    mode http
```

```
    http-request set-header X-Forwarded-Proto https if {  
ssl_fc }
```

```
    default_backend be_web
```

PEM обычно содержит certificate chain и private key

TLS нужно проверять как конфигурацию и как соединение

Private key должен иметь ограниченные права

Проверка конфигурации: `sudo haproxy -c -f /etc/haproxy/haproxy.cfg`

Проверка TLS: `openssl s_client -connect lb01:443 -servername site.example.test`

SNI помогает выбирать сертификат для конкретного имени

При нескольких сайтах Host Header может направлять site1 и site2 в разные backend

HTTPS к backend может быть обязательным

Вариант 1: Client HTTPS → HAProxy → Backend HTTP

Допустимо только в доверенной внутренней сети по принятой политике

Вариант 2: Client HTTPS → HAProxy → Backend HTTPS

Пример server: ssl verify required ca-file

/etc/haproxy/backend-ca.pem check

verify none не следует показывать как production-рекомендацию

Один HAProxy тоже может стать SPOF

Если lb01 отказал, client01 не попадет ни на web01, ни на web02

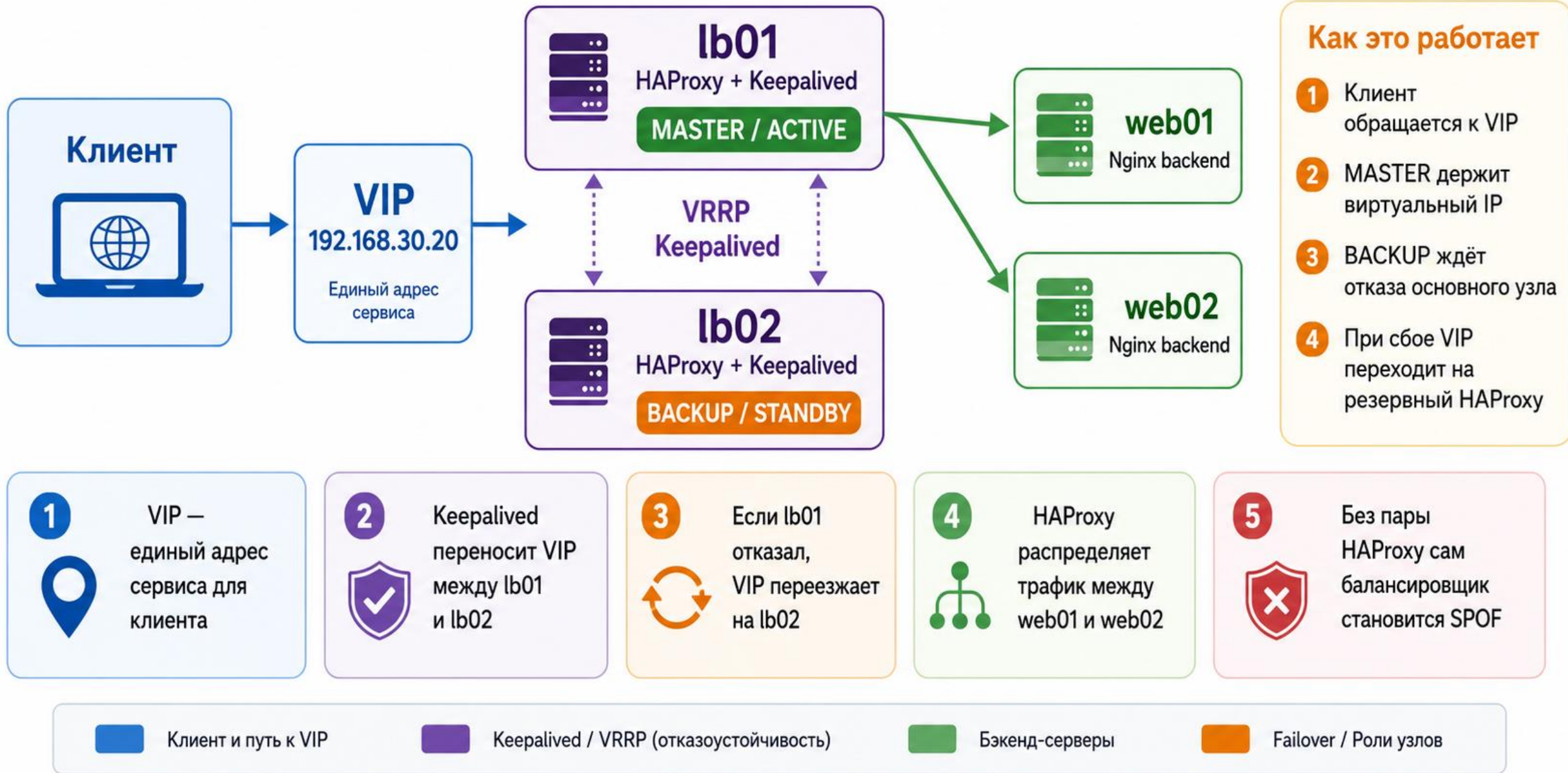
Keepalived и VRRP могут переносить VIP между lb01 и lb02

VIP скрывает активный балансировщик от клиента

Keepalived должен отслеживать не только узел, но и HAProxy process или local health check

Ошибочная общая конфигурация на обоих узлах не исправляется VIP

Схема: HAProxy и обзор Keepalived/VIP



VRRP и VIP зависят от среды

Виртуальная среда может ограничивать multicast VRRP

Может потребоваться gratuitous ARP и корректное MAC learning

В VirtualBox важны promiscuous mode и разрешение перемещения VIP

В облаке влияют security groups и правила на floating IP

Перед лабораторией Keepalived нужно проверять в выбранной среде

Пара HAProxy требует синхронизации конфигураций

Ib01 и Ib02 должны иметь одинаковые frontend, backend, ACL и logging settings

Сертификаты и private keys должны быть доставлены безопасно

Git хранит версию конфигурации, но нужен процесс доставки и применения

Перед reload выполняется haproxy -s на каждом узле

Иначе балансировщики могут обрабатывать трафик по-разному

Проверка конфигурации обязательна до reload

Сначала сделать backup: `sudo cp -a /etc/haproxy/haproxy.cfg /etc/haproxy/haproxy.cfg.bak-$(date +%F-%H%M)`

Затем отредактировать конфигурацию

Проверка: `sudo haproxy -c -f /etc/haproxy/haproxy.cfg`

Ожидаемый результат: Configuration file is valid

Только после этого выполнять reload

reload обычно безопаснее restart

Reload применяет новую конфигурацию с минимальным разрывом соединений, если сервис это поддерживает

Команда: `sudo systemctl reload haproxy`

Проверка статуса: `systemctl status haproxy`

Проверка журналов: `journalctl -u haproxy -b --since '-5 min'`

Функциональная проверка: `curl http://192.168.30.10/`

HAProxy logs нужны для эксплуатации

option httplog включает HTTP-ориентированный формат журнала

HAProxy может писать в /dev/log через syslog

На Debian часто требуется корректная настройка rsyslog или journald

Команда просмотра unit: journalctl -u haproxy -b

Журналы должны ротироваться и отправляться в централизованный сбор

7. Мониторинг HAProxy через Zabbix

Zabbix должен проверять не только процесс HAProxy

Нужны проверки listening ports 80 и 443

Нужна пользовательская HTTP-проверка через балансировщик

Нужен статус backend web01 и web02

Нужно различать потерю резервирования и полный отказ сервиса

Минимальные Zabbix Items для lb01

HAProxy service active: systemd unit state

TCP 80 listening: local port check

TCP 443 listening: local port check, если включен HTTPS

Frontend HTTP response: web scenario или HTTP agent

Backend UP count: значение из stats socket, stats page
или userparameter

HAProxy config age или checksum: контроль
неожиданного изменения

Zabbix triggers должны различать деградацию и отказ

1 из 2 backend DOWN — service работает, но резервирование потеряно

Severity: Average или High по политике

2 из 2 backend DOWN — service unavailable

Severity: Disaster

Отдельный trigger нужен для HAProxy service down и для недоступности пользовательского HTTP

Stats page не должна быть единственной проверкой

Stats page может быть недоступна из-за firewall или management frontend

Она может иметь неверные credentials

Ее недоступность не всегда равна пользовательскому отказу сервиса

Нужны независимые проверки: systemd, port, user-facing HTTP и backend state

Stats полезна, но не заменяет прикладную проверку

Общие зависимости остаются возможными SPOF

Database может быть единой точкой отказа для web01 и web02

Redis, NFS, DNS, storage и authentication также могут быть общими зависимостями

Если shared DB отказала, оба web backend могут стать бесполезны

Zabbix должен показывать не только HAProxy, но и зависимости приложения

Архитектура availability должна охватывать весь сервис, а не только web pool

Лабораторный сценарий HAProxy

Установить Nginx на web01 и web02

Разместить разные страницы и /health

Установить HAProxy на lb01

Создать global, defaults, frontend и backend

Настроить roundrobin, HTTP health check, inter, fall, rise и timeouts

Проверить haproxy -c, reload и серию curl с Connection: close

Лабораторные отказы и восстановление

Остановить Nginx на web01 и увидеть web01 DOWN

Проверить, что запросы продолжают идти на web02

Вернуть web01 и дождаться rise

Сделать /health HTTP 500 и проверить HTTP health check

Остановить оба backend и получить HTTP 503

Сохранить проверенную конфигурацию HAProxy в Git

Приемочная матрица лабораторной работы

haproxy -c: configuration file is valid

ss -lntp: HAProxy слушает ожидаемый IP и порт

curl web01 и web02 напрямую: разные ответы backend

серия curl к lb01: ответы распределяются

один backend down: сервис отвечает через второй backend

оба backend down: клиент получает HTTP 503

Типовые ошибки HAProxy

Не указан mode http, но используются HTTP-функции
bind *:80 случайно открыл сервис на лишних
интерфейсах

TCP check считает backend UP при HTTP 500

Приложение доверяет поддельному X-Forwarded-For
от клиента

Stats page доступна из пользовательской сети

После reload не проверены logs и пользовательский
HTTP-запрос

Итоги лекции

HAProxy может быть reverse proxy и load balancer
одновременно

Для HTTP-лаборатории нужен mode http и
полноценный HTTP health check

Round Robin, Least Connections, weights, sessions и
health endpoint влияют на реальное распределение
X-Forwarded-For, X-Forwarded-Proto, stats и TLS требуют
доверенной и защищенной конфигурации

Отказоустойчивость включает backend, сам HAProxy,
общие зависимости и мониторинг

Контрольные вопросы

Чем Reverse Proxy отличается от Load Balancer?

Почему для HTTP-проверок нужен mode http?

Чем TCP health check отличается от HTTP health check?

Почему curl-проверка Round Robin может искажаться из-за Keep-Alive?

Почему нельзя доверять X-Forwarded-For от внешнего клиента?

Какие Zabbix triggers нужны для деградации и полного отказа HAProxy-сервиса?